

Chapter 3:

Decidability and Decidable Problems

Decidable Languages

Recall that:

A language L is **Turing-Acceptable**
if there is a Turing machine M
that accepts L

Also known as: *Turing-Recognizable*

or

Recursively-enumerable
languages

For any string w :

$w \in L \implies M$ halts in an accept state

$w \notin L \implies M$ halts in a non-accept state
or loops forever

What is Decidability

- A problem is called decidable, when there is a solution to that problem and also can construct algorithms corresponding to that

Example of Decidable Problem

- Find all the odd numbers in the range from 1 to 50.
- For this problem, we can easily find a solution by constructing an algorithm.
- In terms of Turing Machine (TM), if a problem is decidable, then the Turing machine halts whether or not it accepts its input.

- In terms of finite automata (FA), decidable refers to the problem of testing whether a **deterministic finite automata** (DFA) accepts an input string. A decidable language corresponds to algorithmically solvable decision problems

Definition:

A language L is **decidable**
if there is a Turing machine (**decider**) M
which accepts L
and halts on **every input** string

- **Decidability**

For the Recursive Language

- A language 'L' is said to be recursive if there exists a Turing Machine which will accept all the strings in 'L' and reject all the strings not in 'L'.
- The TM will halt every time and give an answer either accepted or rejected for each and every input.

For any string w :

$w \in L \implies M$ halts in an accept state

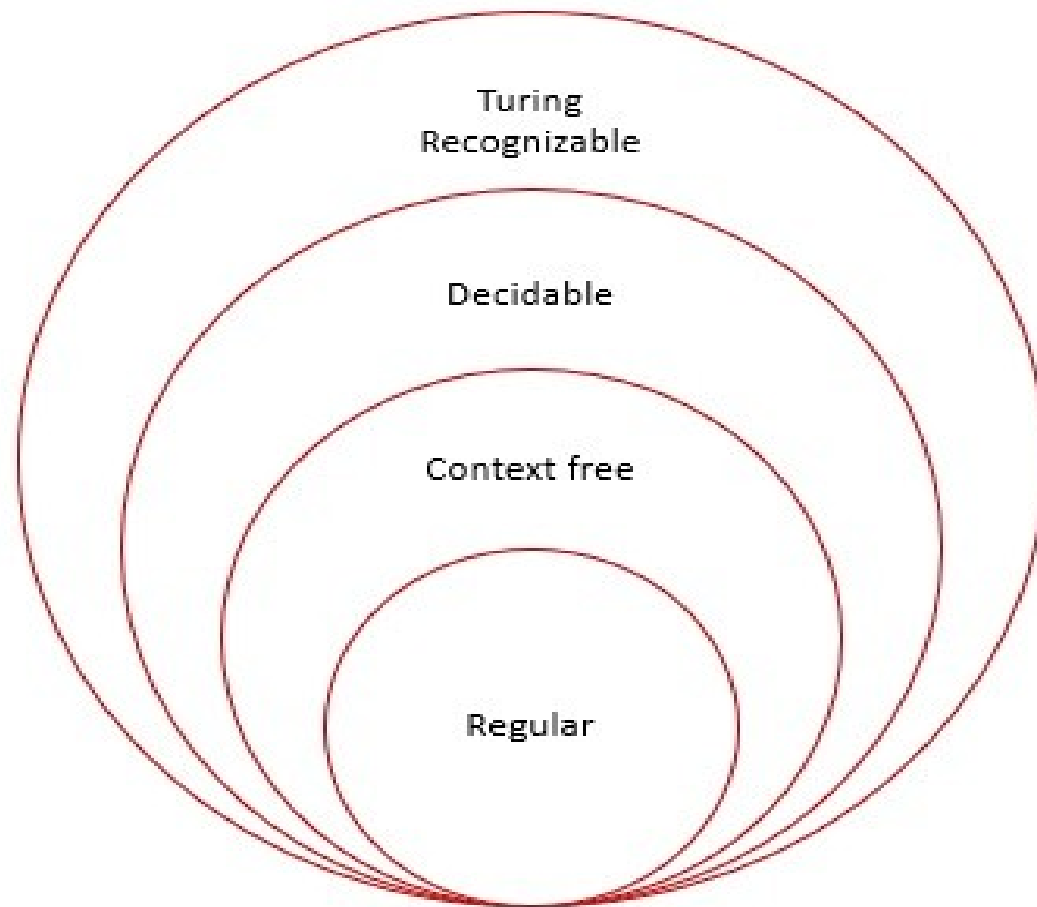
$w \notin L \implies M$ halts in a non-accept state

Every decidable language is Turing-Acceptable

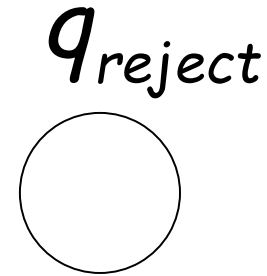
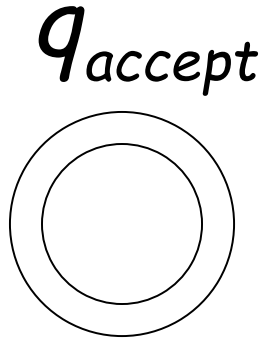
Recursively Enumerable languages –

- A language 'L' is said to be recursively enumerable if there exists a TM which **accepts** and **halt** for all input in 'L'.
- But may or may **not halt** for all input, which are not in 'L'.

- A Language 'L' is decidable if it is a recursive language.
- All decidable languages are recursive languages and vice versa.
- The diagram given below explains the decidable language –



Sometimes, it is convenient to have Turing machines with single accept and reject states

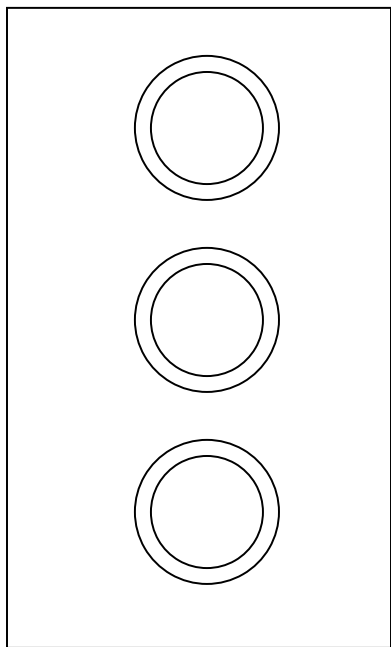


These are the only halting states

That result to possible
halting configurations

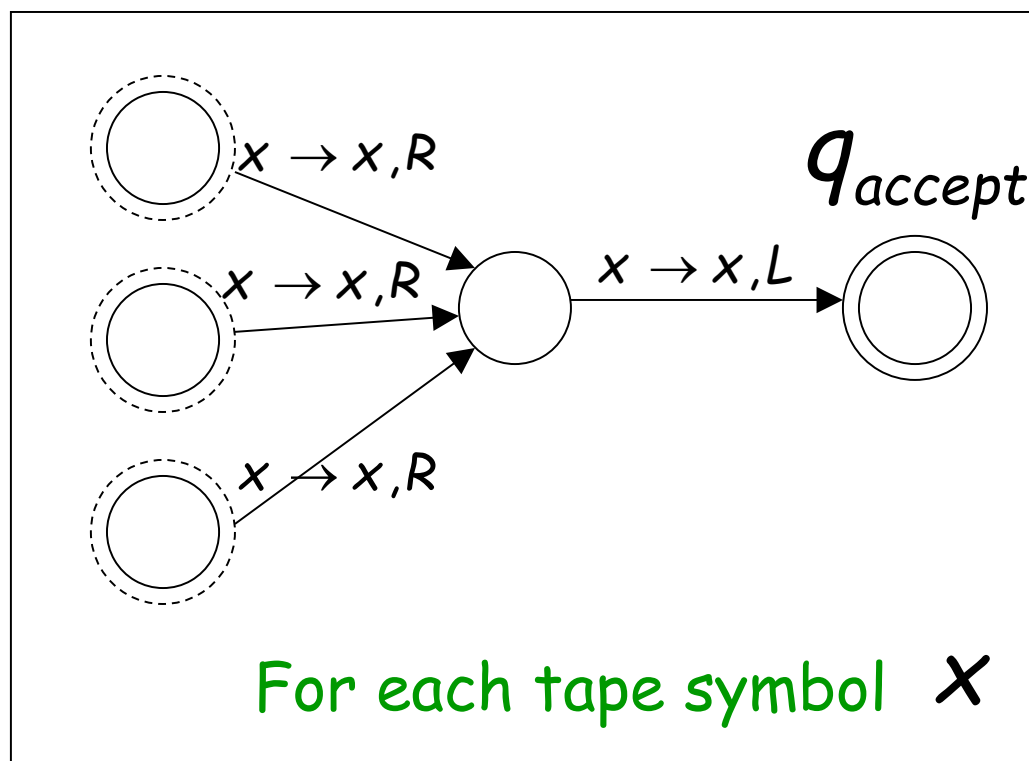
We can convert any Turing machine to have single accept and reject states

Old machine



Multiple
accept states

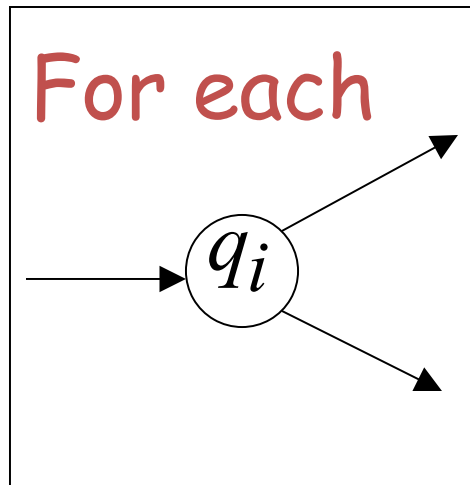
New machine



One accept state

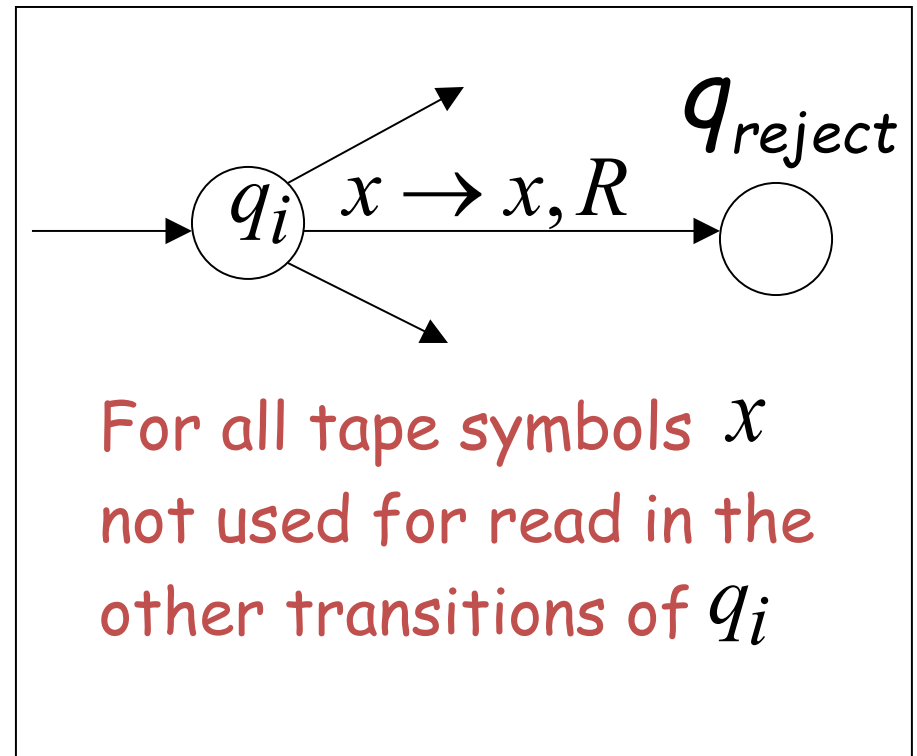
Do the following for each possible halting state:

Old machine



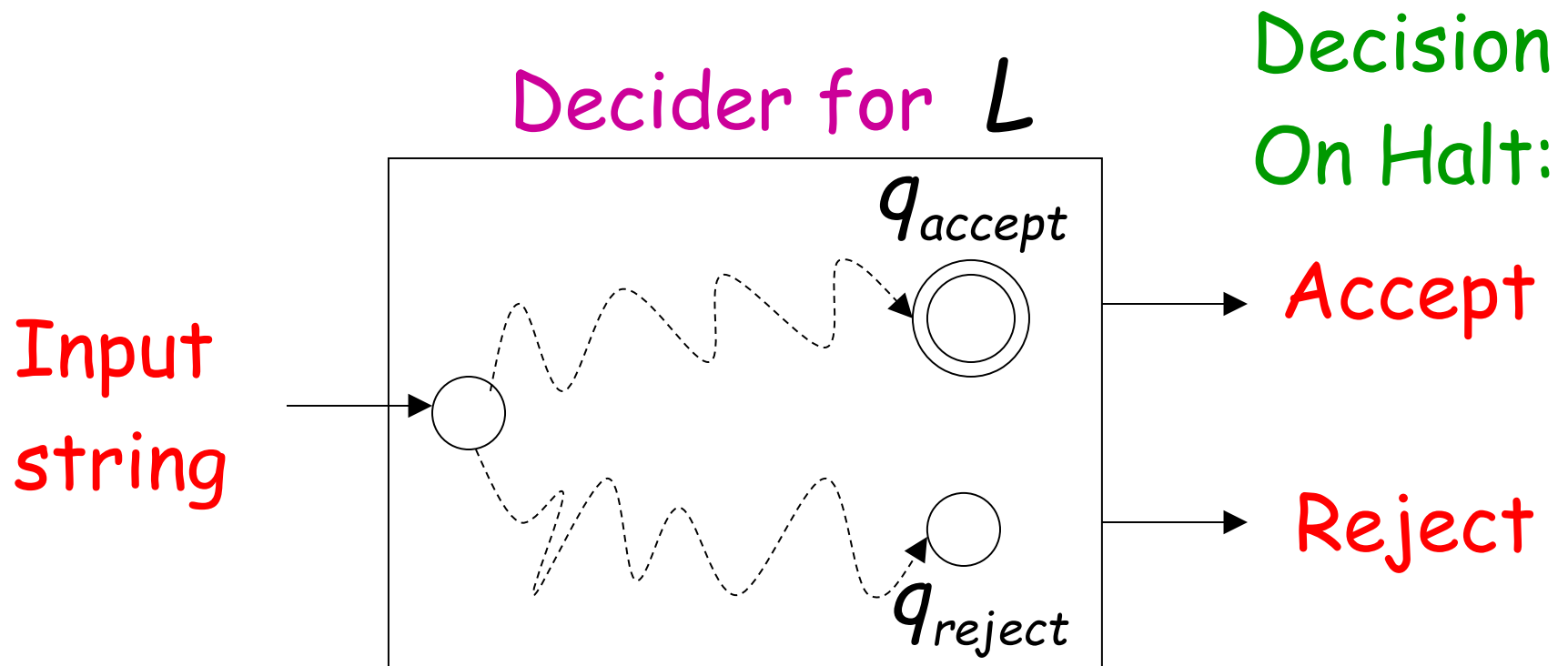
Multiple
reject states

New machine



One reject state

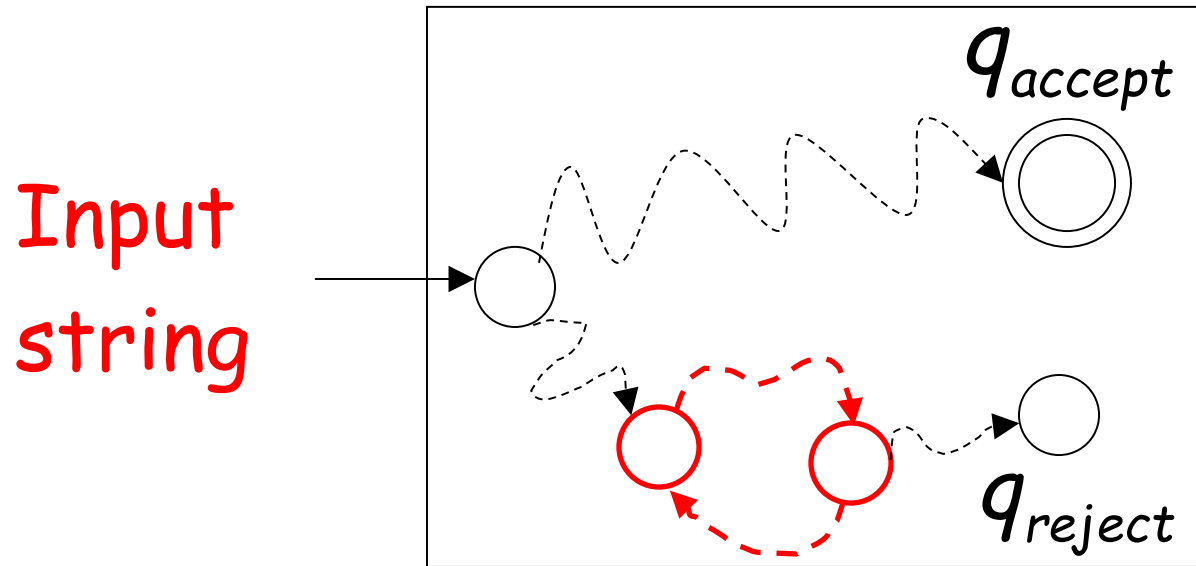
For a decidable language L :



For each input string, the computation halts in the accept or reject state

For a Turing-Acceptable language L :

Turing Machine for L



It is possible that for some input string the machine enters an infinite loop

Problem: Is number x prime?

Corresponding language:

PRIMES: $\{2, 3, 5, 7, \dots\}$

We will show it is decidable

Decider for *PRIMES* :

On input number x :

Divide x with all possible numbers
between 2 and $\sqrt{x}$

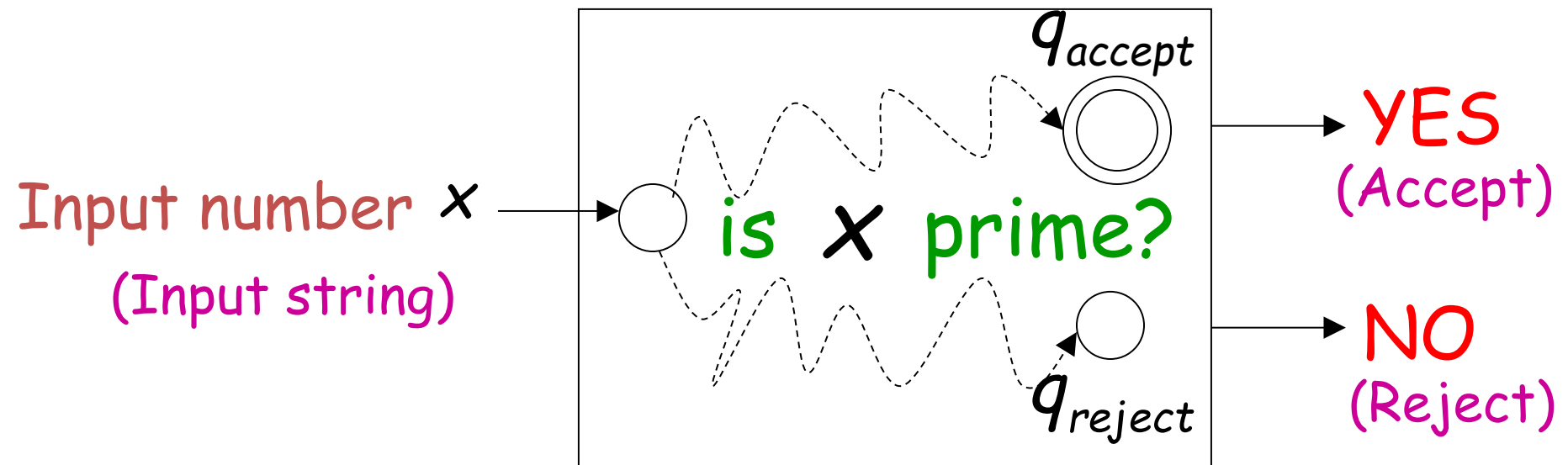
If any of them divides x

Then reject

Else accept

the decider for the language
solves the corresponding problem

Decider for *PRIMES*



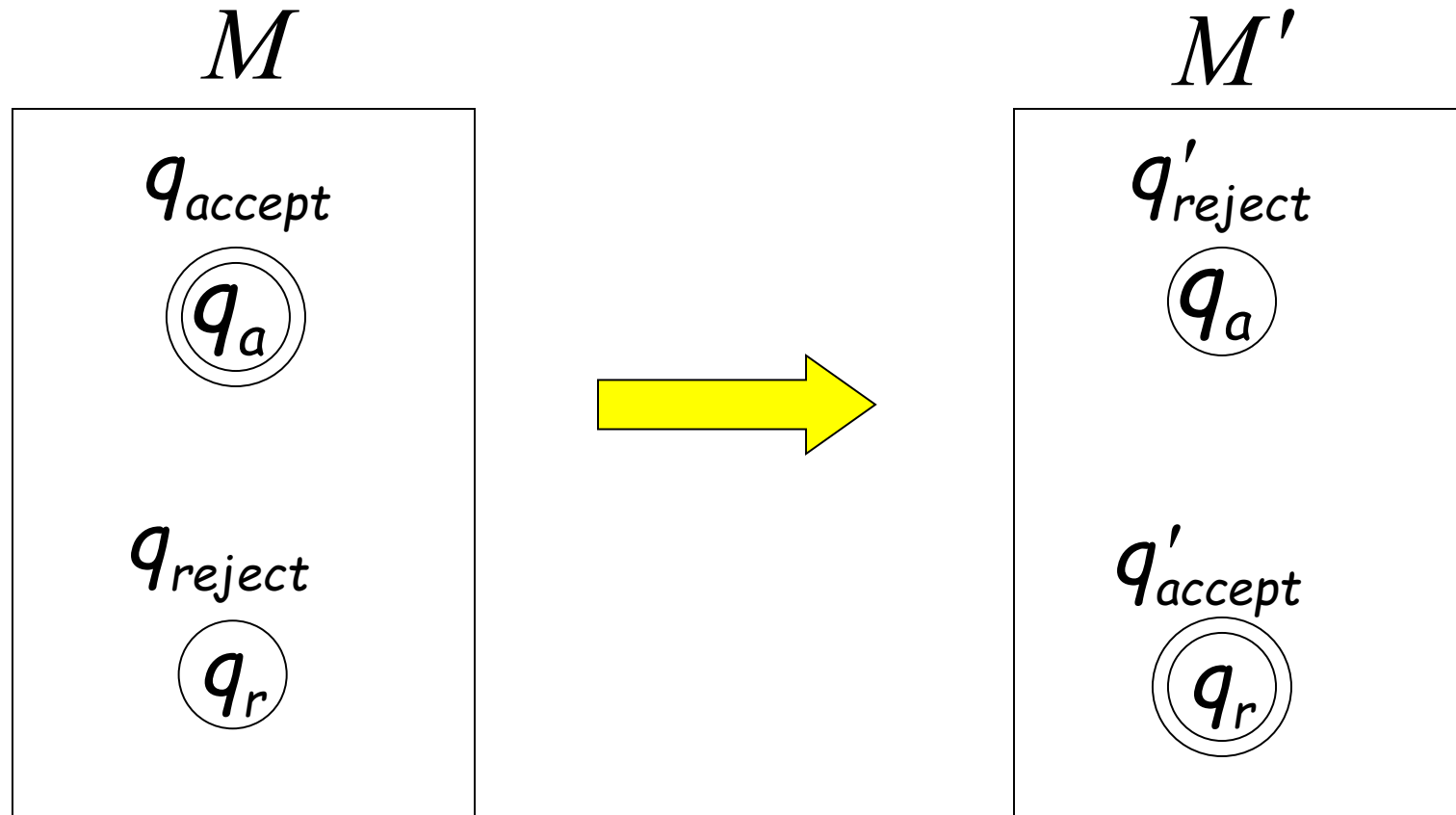
Theorem:

If a language L is decidable,
then its complement $\bar{L}$ is decidable too

Proof:

Build a Turing machine M' that
accepts $\bar{L}$ and halts on every input string
(M' is decider for $\bar{L}$)

Transform accept state to reject and vice-versa



Turing Machine M'

On each input string w do:

1. Let M be the decider for L
2. Run M with input string w
 - If M accepts then reject
 - If M rejects then accept

Accepts $\bar{L}$ and halts on every input string

Which Languages are decidable?

- There are 4 types of Languages which we will define further:
 - Language A_{DFA}
 - Language A_{NFA}
 - Language A_{CFG}
 - Language A_{TM}

Acceptance Problem for DFA

Given a DFA D and the input string w , determine if D accepts w .

- Is this problem solvable?
- Let's turn this problem into a language and build a TM to decide it.

$$A_{\text{DFA}} = \{ \langle D, W \rangle \mid D \text{ is a DFA that accepts } W \}$$

A_{DFA} is the language of all strings that represent a valid encoding of all pairs D and W , where D is an encoding of some DFA, w is the string, and D accepts it.

The machine to decide A_{DFA} is :

$M = \text{"On input } \langle D, W \rangle \text{ where } D \text{ is a DFA and } w \text{ is a string"}$

- 1. Simulate D on w**
- 2. Accept if the simulation ends in the accept state,
reject otherwise.**

Theorem : A_{DFA} is decidable

Proof: The TM M constructed above decides A_{DFA}
 M halts on every input, since a DFA always halts,
so the simulator will halt too.

Acceptance Problem for NFSA's

- Given A NFA N and the input string w , determine if N accepts w .

- The language

$$A_{\text{NFA}} = \{ \langle N, w \rangle \mid N \text{ is a NFA that accepts } w \}$$

Proof: Convert N into the representation equivalent to DFA (such algorithm exists), and run the machine for A_{DFA} , on $\langle D, w \rangle$

Emptiness Problem for DFAs is decidable

- Given a DFA D , determine if D accepts any string at all.

i.e, if $L(D) = \emptyset$

- The language

$$E_{\text{DFA}} = \{ \langle D \rangle \mid D \text{ is a DFA and } L(D) = \emptyset \}$$

Proof: Algorithm (or TM) to decide it:

Given a DFA $\langle D \rangle$, can we go from the initial state to a final state ?

If so, the DFA could generate some string.

$\Rightarrow$ Is there any final state reachable from the initial state?

$\Rightarrow$ A graph problem.

- **Repeat until no new state get marked.**
mark any state where there is a transition
from an already marked state to a previously
unmarked state.

Check to see if any final state got marked.

If a final state is marked, the language is not empty. So,
reject.

Else

The language is empty, So, reject.

Theorem : Equivalence Problem for DFAs is decidable

Given two DFAs A and B, determine if they recognize the same language.

i.e: $L(A) = L(B)$

The language

$EQ_{DFA} : \{ \langle A, B \rangle \mid A \text{ and } B \text{ are DFAs and } L(A) = L(B) \}$

Is decidable.

Proof: Let C be the Symmetric difference between A and B.
(Any thing in A or B, but not in both).

$$C = A \setminus B \cup B \setminus A$$

$$C = (A \cap B') \cup (A' \cap B)$$

Note: If $A=B$, then the symmetric difference is $\emptyset$.

Given

A = DFA to accept $L(A)$

B = DFA to accept $L(B)$

We know how to construct DFA,

$L(A)$

$L(A) \cap L(B)$

$L(A) \cap L(B)$

Build DFA C to accept the symmetric difference.

- Use the TM from the previous theorem ($E_{\text{DFA}} = \text{Empty language}$) to test.

Summary:

- Construct a TM
- Input: $\langle A, B \rangle$ (two DFAs)
- Construct a ~~DFA~~ C to ~~accept~~

$$L(C) = (L(A) \cap L(B)) \cup (L(A) \cap L(B))$$

- Use the previous TM to test whether the language that C accepts is empty.
-
- Accept if so; Reject otherwise.

Emptiness Problem of CFG

- Given a CFG, is the language it generates empty?
- This is a decidable problem
- $E_{CFG} = \{ \langle G \rangle \mid G \text{ is a CFG and } L(G) = \emptyset \}$
- E_{CFG} is a decidable language.

- To test whether $L(G)$ is empty, we need to test whether G can generate a string of terminals
- Determining for each variable whether that variable can generate a string of terminals
- When the algorithm determines that a variable can generate a string of terminals the algorithm marks that variable.
- The algorithm starts by marking first all terminals. Then it marks variables that have on their right hand side in some rules only terminals, i.e., marked symbols, and so

Algorithm (a marking Algorithm)

input : A CFG “G”

1. Mark all terminal symbols of G
2. Repeat until no new variable get marked:
Mark any variable A where G has a rule
 $A \rightarrow u_1 u_2 \dots u_k$ has already been marked
3. If the start symbol of G is not marked, accept;
otherwise reject

- Example:

Consider the following grammar:

S ABCD

A BCA

A xyz

B CA

B AB

B BBBw

C CB

C ww

D ~~DD~~

B BD

Is $L(G) = \emptyset$?

→

→

→

→

Solution:

- Mark A everywhere it occurs since it generates all terminals
- Mark C, since it generates string of terminals

Consider:

$B \rightarrow CA$

Since both C and A generate string of terminals, B can also generate string of terminals. So mark B.

Can D generate string of terminals? No.

D cannot be marked.

S cannot be marked since D cannot be marked.

So, $L(G)$ is empty.

Acceptance problem of CFG :

- Checking if a CFG generates a particular string

Problem: For a given CFG grammar (N, T, P, S) and string $w \in \Sigma^*$, does G generates w (i.e., is $S \Rightarrow^* w$ true?)

Language:

$A_{CFG} = \{ \langle G, w \rangle \mid G \text{ a CFG that generates string } w \}$

- A_{CFG} is Decidable

Fact :

- If G is a CFG in Chomsky normal form then for any $w \in L(G)$ where $|w| = n$, exactly $2n-1$ steps are required for any derivation of w .

The TM that decide A_{CFG}
is:

S= "On input $\langle G, w \rangle$, where G is a CFG and w is a string:

1. Convert G to an equivalent grammar in Chomsky normal form
2. List all derivations with $2n-1$ steps, except if $n \leq 1$;
for $n \leq 1$, list all derivations with 1 step
3. If any of the derivations listed above generates w , accept; if not reject

Closure Properties of Decidable Languages

Theorem (Closure Properties of Decidable Languages)

- The class of decidable languages is closed under intersection, union, complement, concatenation and Kleene star.
- If L_1 and L_2 are decidable languages, then $L_1 \cap L_2$, $L_1 \cup L_2$, L_1^c , $L_1.L_2$ and L_1^* are decidable too.

Proof:

Closure of Decidable Languages under Intersection

- Let L_1 and L_2 be decidable. We show that $L_1 \cap L_2$ is decidable too.
- Let M_1 be a decider for L_1 and M_2 be a decider for L_2 .
- Consider a 2-tape TM M :
 - "On input x :
 1. copy x on the second tape
 2. on the first tape run M_1 on x
 3. if M_1 accepted then goto 4. else M rejects
 - $M =$ 4. on the second tape run M_2 on x
 5. if M_2 accepted then M accepts else M rejects."
 - **The machine M is a decider and it accepts a string x iff both M_1 and M_2 accept x .**

Proof: Closure of Decidable Languages under Complement

- Let L_1 be a decidable language. We show that L_1' is decidable too.
- Let M_1 be a decider for L_1 .
- Consider a TM M :

”On input x :

1. run M_1 on x

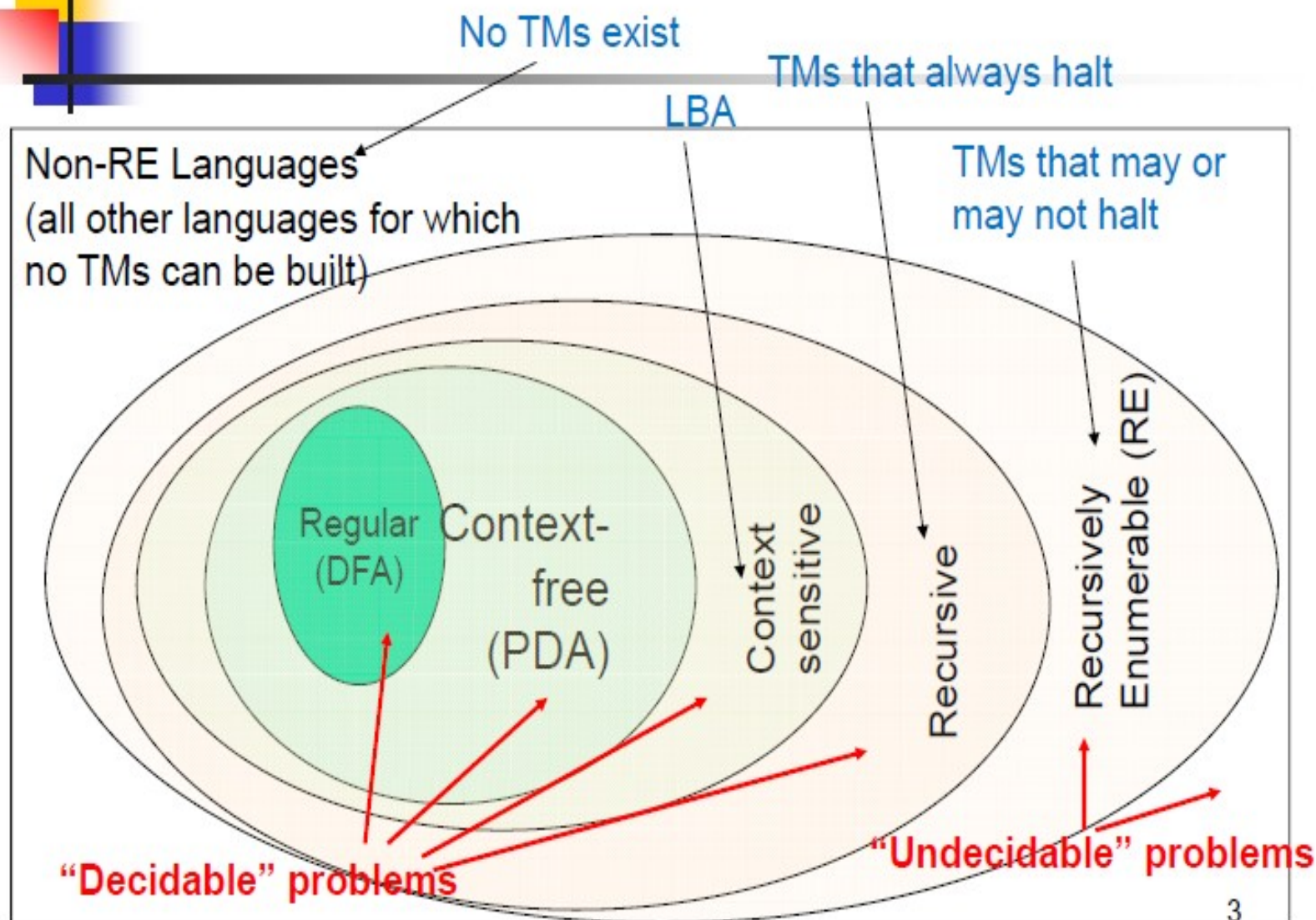
2. if M_1 accepted then M rejects else M accepts.”

• **M** The machine M is a decider, and it accepts a string x iff M_1 rejects x . Hence M decides L_1'

undecidable problems

- The problems for which we can't construct an algorithm that can answer the problem correctly in the finite time are termed as Undecidable Problems.
- A problem is undecidable if there is no Turing machine that will always halt in a finite amount of time to answer as 'yes' or 'no'.

Recursive, RE, Undecidable languages



The examples of undecidable problems:

1. Given **two context-free languages**, there is no Turing machine that will always halt in a finite amount of time and give an answer whether **two context-free languages are equal or not**.
2. Equivalence For **Context-Free Grammars** is undecidable.

$$EQ_{CFG} = \{ \langle G1, G2 \rangle \mid G1 \text{ and } G2 \text{ are CFGs and } L(G1) = L(G2) \}$$

3. Whether a CFG generates all the strings or not?

As a CFG generates infinite strings, we can't ever reach up to the last string and hence it is Undecidable.

4. Ambiguity of CFG?

There exist no algorithm which can check whether for the ambiguity of a CFL.

Undecidable Problems related to Turing machine:

- **Membership** problem of a Turing Machine?
- **Finiteness** of a Turing Machine?
- **Emptiness** of a Turing Machine?
- Whether the language accepted by Turing Machine is regular or CFL?

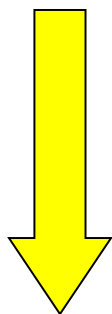
Undecidable Languages

An undecidable language has no decider:
each Turing machine that accepts L
does not halt on some input string

There is a language which is
Turing-Acceptable and undecidable

there is a language L :

- $\overline{L}$ is **not** Turing-acceptable
(not accepted by any Turing Machine)
- L is Turing-acceptable



the complement of a
decidable language is decidable

Therefore, L is undecidable

Non Turing-Acceptable $\overline{L}$

Turing-Acceptable L

Decidable

Non Turing-Acceptable

$\overline{L}$

Turing-Acceptable

Decidable